



الجمهورية الجزائرية الديمقراطية الشعبية
وزارة التعليم العالي والبحث العلمي
جامعة وهران للعلوم والتكنولوجيا محمد بوضياف
كلية الرياضيات والاعلام الالي

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur Et de la Recherche Scientifique
Université des Sciences et de la Technologie d'Oran Mohamed BOUDIAF
Faculté des Mathématiques et Informatique

Département : Informatique

Mémoire de fin d'études

A Mobile Application for Personalized Medication Safety with an AI assistant

Pour l'obtention du diplôme
de **Licence**

Domaine : **Mathématiques – Informatique**

Filière : **Informatique.**

Spécialité : **Ingénierie des Systèmes d'Information et du Logiciel**

Présenté le :

Par :

-

Soussi Yacine

Maghraoui Nadji

-

1.

Jury	Nom et Prénom	Grade	Université
Examineur	Said Khelifa	MCB	USTO-MB
Encadrant	Goumidi mohammed abdessamad	MAB	USTO-MB
Co-Encadrant	Henkouch Djamel	MCB	USTO-MB

Acknowledgements

It is with profound gratitude that we express our sincere thanks to everyone who contributed, directly or indirectly, to the completion of this thesis. First and foremost, we thank Allah for His infinite grace, strength, and guidance throughout this academic journey.

Our warmest thanks go to our parents for their unconditional love, constant sacrifices, and invaluable moral support. Their presence and prayers have been our greatest source of motivation. We also extend our deep appreciation to all our professors throughout these university years for their knowledge, availability, and commitment to providing us with the foundation necessary for our professional growth.

We would like to especially thank our supervisor, Mr. Goumidi ,and Co supervisor Mr. Hankouche for their precious advices, patience, and benevolent guidance throughout this project. Our thanks also go to our examiner ,Mr Khelifa for the time dedicated to evaluating this work and for his fair and objective perspective. Finally, we warmly thank our sisters, brothers, and friends for their moral support, constant encouragement, and comforting presence during this adventure.

This thesis is the result of collective effort, hard work, and a commitment to excellence. We hope it represents a solid foundation for our professional future.

Abstract

The healthcare sector faces growing challenges in managing patient safety due to the risk of adverse drug events caused by complex treatments and fragmented medical records. This thesis presents **MediCheck**, an intelligent mobile application designed to centralize a user’s medical identity and provide real-time pharmaceutical safety checks.

Developed using the Flutter framework and Firebase, the solution features a Clinical Analysis Engine that utilizes Artificial Intelligence to evaluate drug-to-drug and drug-to-disease interactions. By cross-referencing standardized medical data with personalized user profiles—including allergies and chronic conditions—MediCheck provides natural-language clinical reasoning to explain potential risks. The final result is a functional cross-platform tool that enhances patient autonomy and reduces medication errors in the digital health landscape.

Contents

Acknowledgements	i
Abstract	ii
General Introduction	1
Context	1
Problem Statement	1
Objectives	2
Structure of the Thesis	2
1 State of the Art	3
1.1 Introduction	3
1.2 In-Depth Context: Medication Safety and the Patient Gap	3
1.2.1 The Problem of Adverse Drug Events (ADEs)	3
1.2.2 The Patient as the Final “Check”	3
1.3 Overview of Existing Digital Solutions	4
1.3.1 General Health Trackers and Pill Reminders	4
1.3.2 Telemedicine Platforms	4
1.4 Analysis of Limitations and Identification of the Need	4
1.4.1 The “Logic” Gap	4
1.4.2 Positioning MediCheck: The Intelligent Safety Layer	4
1.5 Conclusion	5
2 Conceptual Design and Requirements Analysis	6
2.1 Introduction	6
2.2 Requirements Gathering and Analysis	7
2.2.1 How We Identified the Project Requirements	7
2.2.2 Functional Requirements (What the App Does)	7
2.2.3 Non-Functional Requirements (How the App Performs)	8
2.2.4 Identification of Actors and Use Case Modeling	8
2.3 Solution Design	10
2.3.1 Dynamic Interactions Modeling (Sequence Diagrams)	10
2.3.2 Static Data Design (Class Diagram)	14

2.4	Conclusion of the Chapter	15
3	Realization and Technical Implementation	16
3.1	Introduction	16
3.2	Software and Technology Environment	16
3.3	Global Architecture and Technological Choices	17
3.3.1	Application Mobile (Frontend)	17
3.3.2	Back End Services	19
3.4	General Project Structure and Functional Implementation	20
3.4.1	General Organization and State Management	20
3.4.2	Authentication Implementation	20
3.4.3	Medical Profile Implementation	23
3.4.4	Interaction Checker and Clinical Analysis Engine	24
3.5	Conclusion of the Chapter	26
	General Conclusion	27

List of Figures

2.1	Use Case Diagram of MediCheck	10
2.2	Sequence Diagram: User Login	11
2.3	Sequence Diagram: Medical Profile	12
2.4	Sequence Diagram: Interaction Analysis	13
2.5	Class Diagram of MediCheck	14
3.1	Flutter Logo	17
3.2	Dart Logo	18
3.3	Android Studio Logo	18
3.4	Firebase Logo	19
3.5	Registration Screen	21
3.6	Login Screen	23
3.7	Medical Info Screen	24
3.8	Interaction Checker and Clinical Analysis Screen	26

General Introduction

Context

Digital transformation is currently redefining the global healthcare sector, with mobile health (mHealth) emerging as a major catalyst for improving access to care and service efficiency. In this dynamic, Algeria has affirmed its commitment to modernizing its healthcare system through digital technology, recognizing the potential of information systems to optimize patient management. While many digital efforts focus on administrative data, a critical pillar of patient safety remains underserved: the prevention of adverse drug events.

This national digital ambition, coupled with the widespread adoption of smartphones, creates a favorable environment for innovative mobile solutions. However, the effective integration of these technologies into daily medication management—particularly in local contexts like Oran—represents a field where targeted improvements are possible. Patients are increasingly demanding autonomy, simplicity, and immediacy in accessing and understanding their health information, making mobile applications a privileged channel for meeting these new needs.

Problem Statement

As healthcare complexity grows, patients face significant challenges in managing multiple medications safely. Traditional methods of checking for drug-to-drug or drug-to-disease interactions often rely on manual research or fragmented consultations with different specialists. This situation is not only time-consuming but can lead to significant clinical risks if a patient lacks a centralized tool to cross-reference their full medical history—including chronic diseases, past surgeries, and allergies—with new prescriptions.

The patient journey is often fragmented: the data from an allergy diagnosis in one clinic might not be available when a new medication is prescribed in another. Patients lack tools that offer real-time visibility into how their unique clinical profile interacts with their treatments. This absence of an integrated, intelligent mobile solution creates a dependency on static information and increases the risk of preventable medication

errors. The challenge, therefore, is to bridge this functional and technological gap with a unified, “smart” assistant.

Objectives

The primary objective of this thesis is the design and development of a functional mobile application, **MediCheck**. This project aims to transform the patient experience by:

- **Standardizing Clinical Profiles:** Allowing users to build a persistent digital medical identity including physical metrics and history.
- **Personalized Interaction Analysis:** Providing a dedicated engine to check for safety risks between medications and existing conditions.
- **AI-Assisted Reasoning:** Integrating an intelligent assistant to translate complex medical data into clear, natural-language clinical explanations.
- **Cross-Platform Accessibility:** Developing a hybrid solution (iOS/Android) using the Flutter framework to ensure a seamless and intuitive user interface.

Structure of the Thesis

This thesis is organized into three main chapters:

- **Chapter 1, State of the Art:** This chapter establishes the clinical and technological context of MediCheck. It identifies the risks associated with adverse drug events and evaluates existing mobile health solutions. The primary focus is defining the “Logic Gap” where current apps fail to provide personalized clinical reasoning.
- **Chapter 2, Conceptual Design and Requirements Analysis:** Details the functional needs, software architecture, and the structure of the clinical database, focusing on UI/UX tailored for health monitoring.
- **Chapter 3, Realization and Technical Implementation:** The MediCheck development phase involved migrating conceptual requirements into a functional mobile solution using the Flutter SDK and Dart for high-performance cross-platform delivery. Firebase was utilized to manage backend infrastructure, specifically for secure authentication and cloud data storage. The system relies on a modular architecture that separates the user interface from business logic and analysis engines to improve scalability. Core implementation efforts focused on medical profile management and the development of an intelligent interaction checker.

Chapter 1

State of the Art

1.1 Introduction

The modern healthcare landscape is increasingly defined by the “informed patient.” As medication regimens become more complex, the risk of adverse drug events—ranging from minor side effects to life-threatening interactions—grows significantly. This chapter establishes the foundation for MediCheck, an application designed to empower individuals with the tools to manage their pharmaceutical safety. We will examine the current challenges in medication management, analyze existing digital health solutions, and identify the gaps that necessitated the creation of an AI-integrated safety engine built with the Flutter framework.

1.2 In-Depth Context: Medication Safety and the Patient Gap

1.2.1 The Problem of Adverse Drug Events (ADEs)

An adverse drug event is any injury resulting from medical intervention related to a drug. These are often preventable and frequently occur due to “polypharmacy” (the use of multiple medications) or a lack of communication between different healthcare providers. For the patient, keeping track of how a new prescription interacts with their existing chronic conditions or allergies is a massive cognitive burden.

1.2.2 The Patient as the Final “Check”

In the traditional care pathway, the patient is the final link in the chain. After leaving the doctor or pharmacy, the responsibility of monitoring symptoms and adhering to safety protocols falls on the individual. However, without professional medical knowledge, patients often struggle to understand the clinical reasoning behind certain

contraindications. There is a clear need for a tool that translates complex clinical data into actionable, personalized insights.

1.3 Overview of Existing Digital Solutions

1.3.1 General Health Trackers and Pill Reminders

Most mHealth (mobile health) applications currently on the market focus on adherence—reminding the user when to take their pills. While useful for habit-forming, these apps are “static”; they rarely analyze what the medication is or how it interacts with the user’s specific physiology.

1.3.2 Telemedicine Platforms

Telemedicine has expanded access to doctors but does not solve the real-time, day-to-day safety check. A patient cannot call a doctor every time they consider taking an over-the-counter painkiller; they need an “on-demand” expert system.

1.4 Analysis of Limitations and Identification of the Need

1.4.1 The “Logic” Gap

Existing solutions are either too simple (reminders) or too complex (medical databases). There is a missing middle: an Expert System that uses Artificial Intelligence to bridge the gap between raw medical data and the user’s personal health context.

1.4.2 Positioning MediCheck: The Intelligent Safety Layer

This panorama highlights a clear need for MediCheck. The project is positioned not just as a tracker, but as a Safety Layer. By utilizing the Flutter framework for a seamless cross-platform experience and Firebase for secure data persistence, MediCheck introduces a specialized clinical engine. It moves beyond “static” lists by using AI to provide personalized, natural-language reasoning for every drug interaction check.

1.5 Conclusion

This chapter has defined the pharmaceutical safety gap and identified the limitations of current mHealth solutions. We have established that while data is available, it is not “intelligent” or “personalized” enough to prevent most medication errors at the patient level. By justifying the need for an integrated AI safety assistant, we lay the groundwork for the technological choices that follow. The next chapter will detail the functional design and the architecture required to bring this intelligent system to life.

Chapter 2

Conceptual Design and Requirements Analysis

2.1 Introduction

This chapter will demonstrate the complete process and identification of our needs and the conceptual techniques that will be implemented in the mobile application MediCheck. This phase is essential for the development process as it will allow us to define the expected functionalities of the system as well as the constraints that should be considered. We begin by identifying both functional and non-functional (such as security and availability) requirements, based on the project objectives and the needs of our end users. This will guide us to define our main services that should be provided by the app, particularly the ability to detect drug and medication interactions and to provide access to reliable medication information. Next, we define the different actors interacting with the system and describe their roles through use case modeling. This step helps clarify how users will interact with the application and establishes a clear functional scope. Finally, we present the overall system design by describing the adopted software architecture. The solution is based on a frontend developed using Flutter and a backend powered by Firebase. We will also be showing the main system interactions using sequence diagrams in order to structure a comprehensive view of how the application operates.

This chapter lays the foundation for the implementation phase by ensuring a clear understanding of the system and the overall structure.

2.2 Requirements Gathering and Analysis

2.2.1 How We Identified the Project Requirements

The identification of requirements for the development of the MediCheck mobile application was based on a pragmatic approach, initiated by a direct observation of the risks associated with self-medication and completed by an analysis of the general context of pharmaceutical safety.

The idea for this project came from the realization of a critical public health issue: many patients, especially those with chronic illnesses or specific allergies, face significant difficulties in identifying whether a non-prescription or newly prescribed medication is safe for them. Traditional medical leaflets are often written in complex and technical language that many users fail to read or understand. This problem allowed us to confront general observations about public health with the specific operational reality of everyday patients.

2.2.2 Functional Requirements (What the App Does)

To keep the app focused and highly effective, we categorized these requirements into a few main pillars that guide the user's journey.

User Authentication & Access

- **Account Creation:** Users must be able to securely register a new account to keep their personal data private.
- **Secure Login:** Returning users need a simple and secure login system to access their saved profiles.

Personal Medical Profile

- **Health Data Input:** Upon setting up their account, users can input critical health details including their age, weight, past surgeries, and known allergies.
- **Profile Management:** Users should be able to update their information at any time if their health status changes.

Multi-Drug Interaction & Risk Checker

- **Multi-Search Functionality:** The system must allow users to search for and select two or more different medications or drugs simultaneously.

- **Real-Time Risk Calculation:** The app must process the selected drugs against each other to calculate the interaction risk.
- **Dynamic Results & Description:** The app must clearly display whether the risk is High or Low, followed by a clear description explaining why the interaction is risky and what the user should watch out for.

2.2.3 Non-Functional Requirements (How the App Performs)

We knew the app would fail if it were not fast, secure, and easy to use. Since we are dealing with sensitive medical data and potentially dangerous drug interactions, we set high standards for how the system should behave. We focused on four main areas.

Security and Privacy

Since users are putting in their private medical history, age, and weight, keeping this data safe is our absolute top priority. We need to make sure all database communication is secure so that no one else can access a user's health profile.

Speed and Performance

When a user is checking two or more drugs, they should not be staring at a loading screen. The search queries and the interaction checks need to happen within a matter of seconds to give users the quick answers they need.

Usability

A major goal was making sure that anyone could use this app, whether they are a student or an older adult managing multiple medications. We kept the layout simple and avoided cluttering the screens with too much medical jargon. The buttons are easy to find and the text is highly readable.

Reliability

The risk assessment descriptions must load accurately every single time. If the database fails to fetch the correct interaction warning, it could lead to misinformation, so the connection between the application and the backend must be as stable as possible.

2.2.4 Identification of Actors and Use Case Modeling

Before mapping out the system's behavior, we need to clearly identify who is using the application and what external systems interact with it. In UML (Unified Modeling

Language), these are called Actors. Once the actors are defined, we can list the Use Cases.

Identification of Actors

For this system, we have identified two main actors:

- **The User (Primary Actor):** This is the patient or the individual using the application. They interact directly with the mobile interface to manage their health profile and check for medication interactions.
- **The Firebase Backend (Secondary Actor):** This is an external system. While it is not a human user, it acts as an actor because our app relies on it to securely authenticate users, store medical profiles, and retrieve medication data.

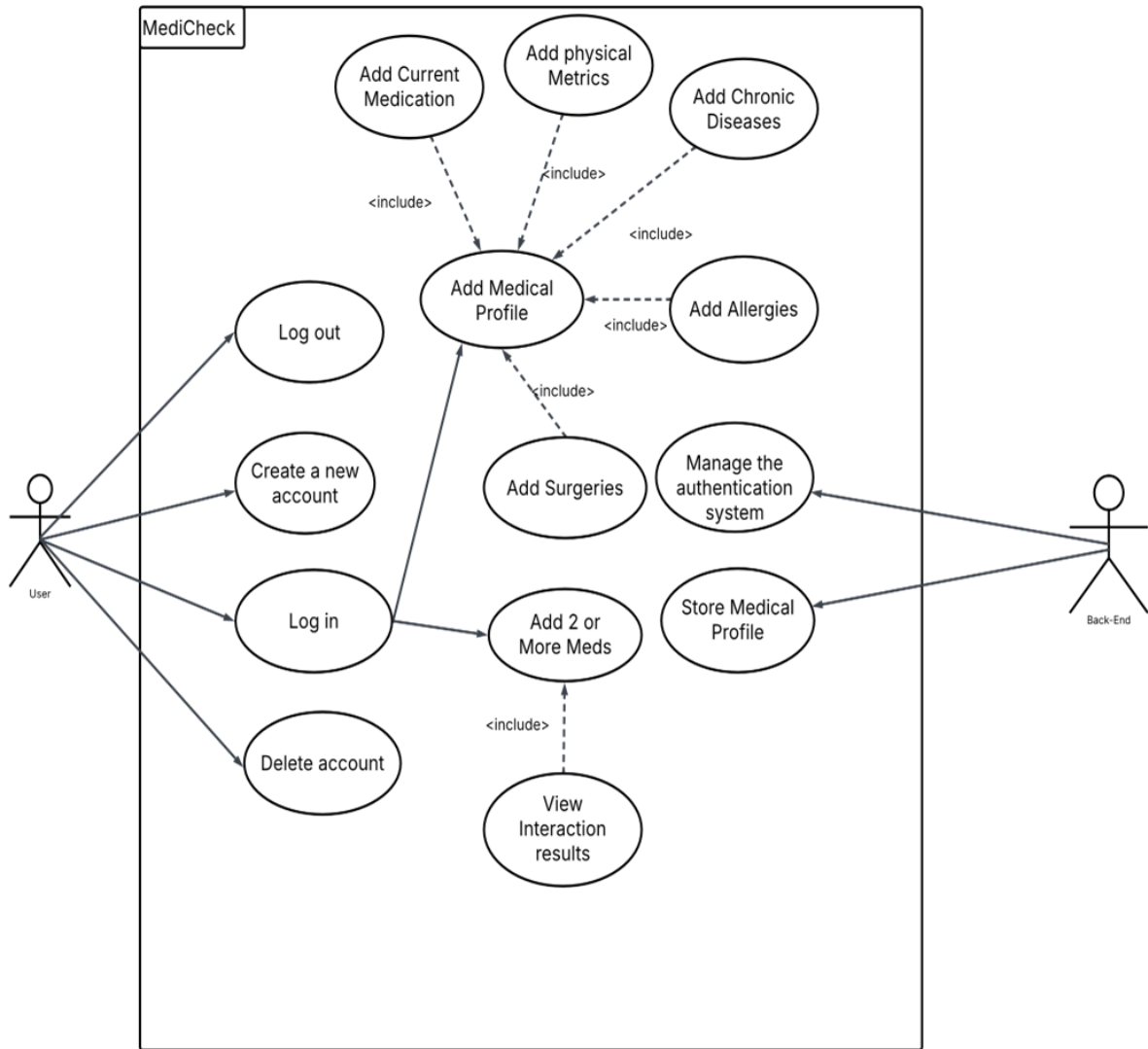
Use Case Diagram Analysis for MediCheck

The Use Case Diagram for the MediCheck application (illustrated in Figure 2.1) highlights the critical interactions between our primary user, the system’s core features, and the backend support.

At the entrance of the system, we find the User. This individual interacts with basic account management features, allowing them to “Create a new account” and securely “Log in” to the application. Once logged in, users can also choose to “Log out” or “Delete their account,” giving them full control over their personal profile.

MediCheck’s core features, accessible once the user logs in, are:

- **Medical Profile Management:** This allows users to “Add Medical Profile,” including:
 - Add Physical Metrics (age, weight).
 - Add Chronic Diseases and Add Surgeries.
 - Add Allergies.
- **Multi-Drug Interaction Check:** The application’s most critical feature. The user can “Add 2 or More Meds” which flows directly into the final step: “View Interaction Results.”

Figure 2.1: Use Case Diagram of **MediCheck**

2.3 Solution Design

This section focuses on translating the identified requirements into a concrete technical solution. It defines the software architecture chosen for MediCheck and provides a detailed breakdown of the system's core components and their dynamic interactions.

2.3.1 Dynamic Interactions Modeling (Sequence Diagrams)

To illustrate the internal operation and interactions between the main components of the architecture during key use scenarios, we present several sequence diagrams below. Each diagram details the chronology of messages exchanged to achieve a specific function.

User Login

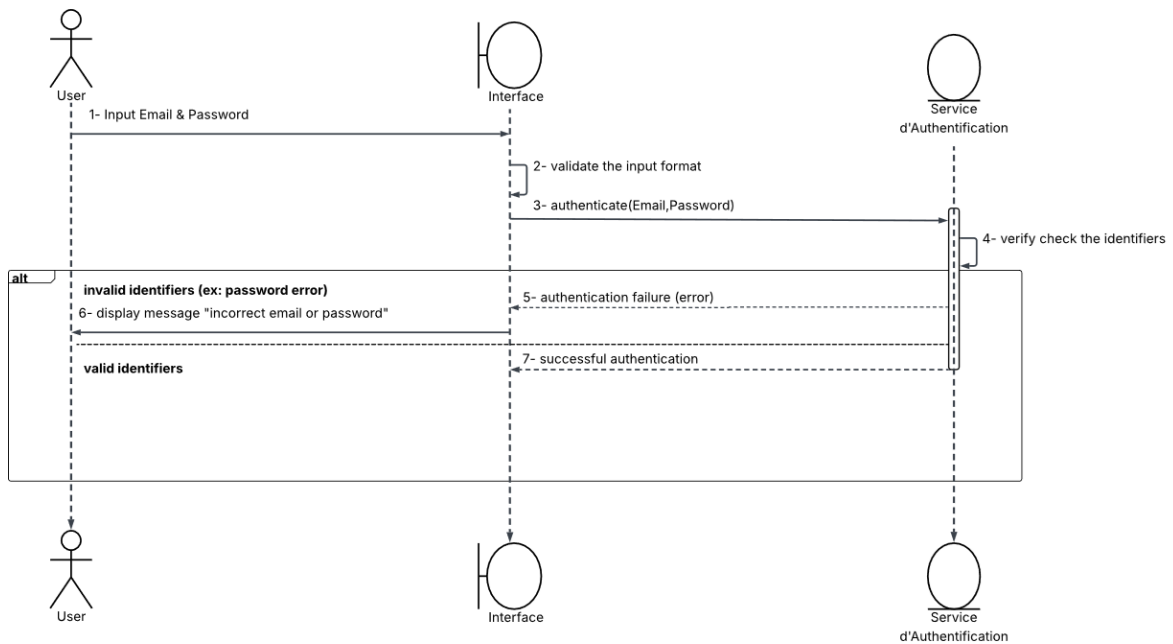


Figure 2.2: Sequence Diagram: **User Login**

Description and Explanation: User Authentication (Firebase Auth)

- **Credential Submission:** The process begins when the User inputs their email and password into the Interface.
- **Format Validation:** Before reaching the server, the Interface performs a local check to validate the input format (e.g., ensuring the email structure is correct).
- **Authentication Request:** The Interface then transmits the credentials to the Service d'Authentification (Firebase) via a secure API call.
- **Verification:** The Service d'Authentification performs an internal check against its encrypted database to verify the identity of the user.
- **Alternative Logic (alt):** The system utilizes an alternative block to manage two distinct outcomes:
 - *Failure Scenario (Invalid identifiers):* If a password error or unknown user is detected, the service returns an error. The Interface then displays a specific “Incorrect email or password” message to provide user feedback.
 - *Success Scenario (Valid identifiers):* Upon a successful match, the service returns a positive acknowledgment.
- **Session Authorization:** Once authenticated, the Interface grants the user access to the secure areas of the application, typically redirecting them to the main dashboard.

Medical Profile

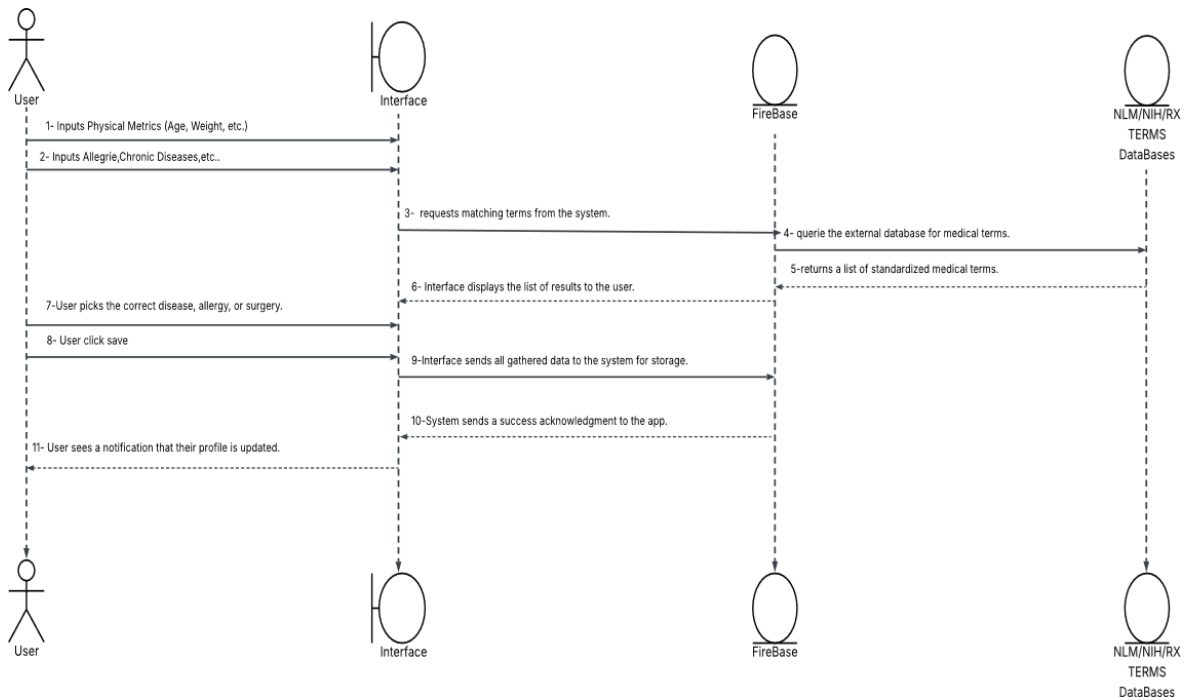


Figure 2.3: Sequence Diagram: **Medical Profile**

Description and Explanation: Medical Profile Management

- **Initialization:** The User begins by populating primary physical metrics (Age, Weight, Height, and Gender) directly into the Interface.
- **Asynchronous Retrieval:** As the User interacts with the search bars for Chronic Diseases, Allergies, or Surgeries, the Interface sends real-time queries to the System.
- **Standardization:** The System cross-references these queries with an External Medical Database (NLM/NIH/RxTerms). This step is crucial as it ensures the user selects standardized medical terminology, which is necessary for accurate risk analysis later.
- **Data Selection:** The Interface populates a dropdown list of suggestions based on the external results, allowing the User to precisely select their specific conditions.
- **Synchronization:** Once the profile is complete and the User clicks “Save,” the Interface transmits the consolidated data packet to the System.
- **Persistence:** The System securely synchronizes this information with Firebase cloud storage.
- **Feedback:** Upon successful verification of the write operation, the System triggers a confirmation message to the Interface, informing the User that their medical profile is now active and ready for the clinical interaction check.

Interaction Analysis

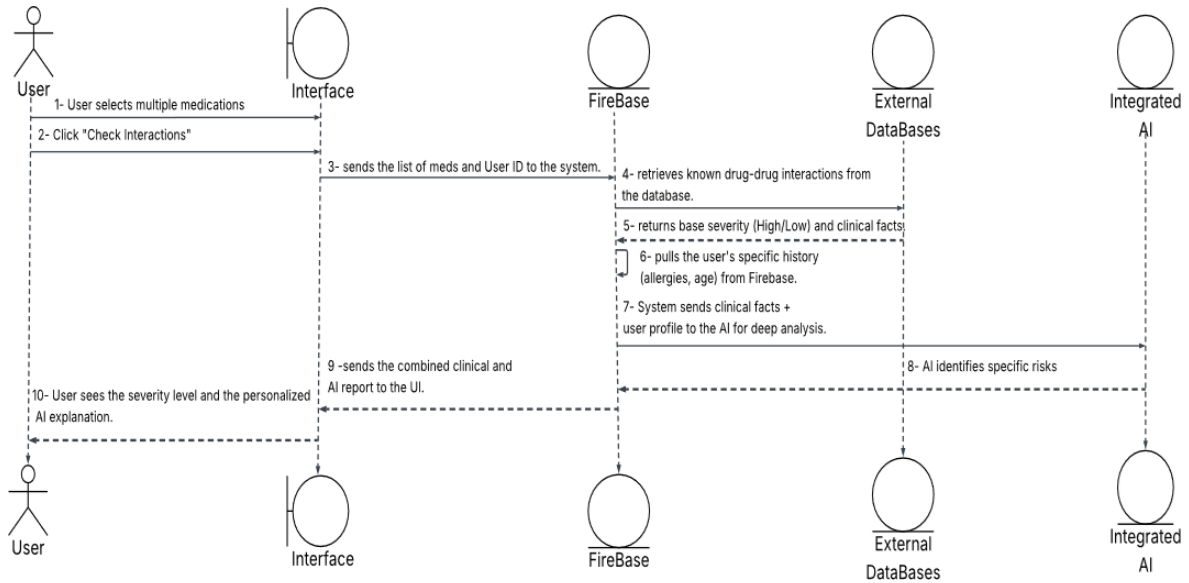
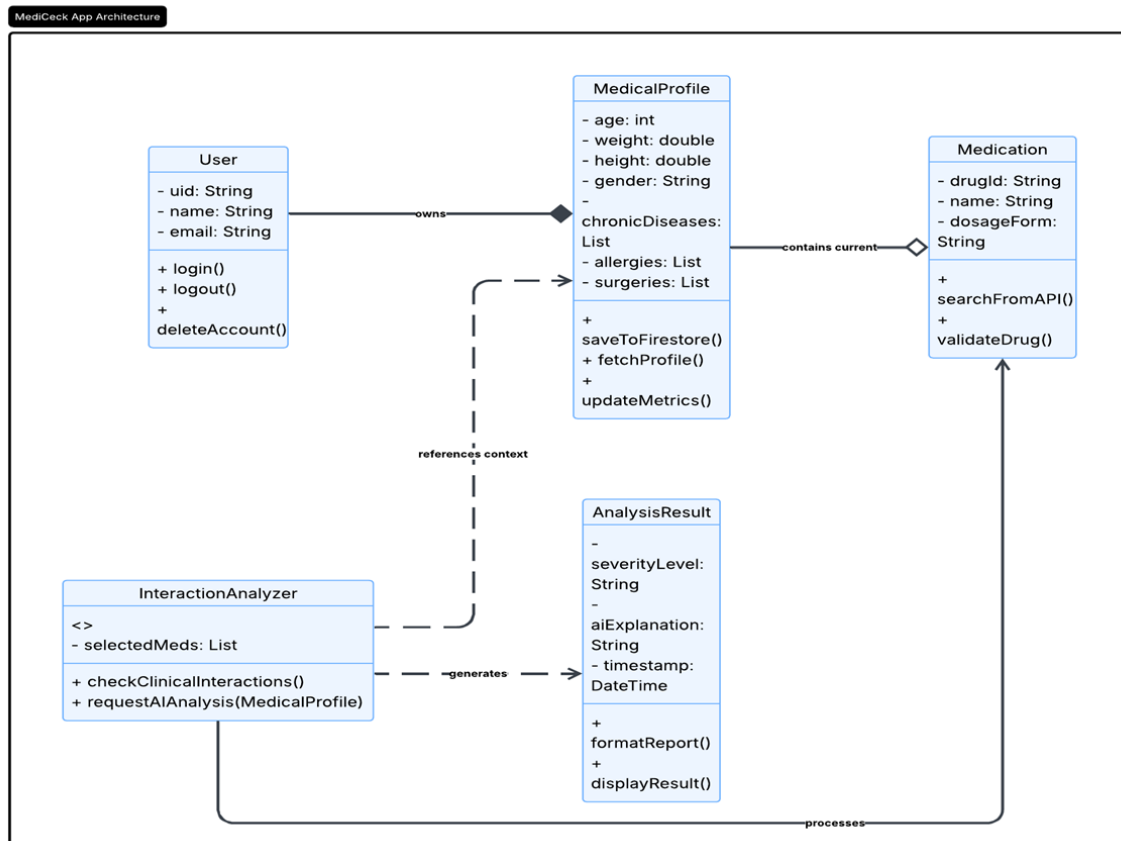


Figure 2.4: Sequence Diagram: **Interaction Analysis**

Description and Explanation: Intelligent Interaction Analysis

- **Clinical Fetch:** Retrieving drug-to-drug conflict data from external pharmaceutical repositories.
- **Contextual Cross-Reference:** Pulling the user's specific health profile (age, allergies) from the internal database.
- **AI Reasoning:** An Integrated AI processes both data sets to generate a personalized risk explanation, which is then delivered to the interface as a comprehensive clinical analysis.

2.3.2 Static Data Design (Class Diagram)

Figure 2.5: Class Diagram of **MediCheck**

Class Diagram Description The class diagram (Figure 2.5) defines the structural organization and data management rules within MediCheck. The architecture focuses on personalizing clinical analysis by correlating the following entities:

- **Identity Management (User):** This class handles session and authentication attributes. It maintains a composition relationship with the profile, meaning that the existence of medical data is strictly dependent on the user account.
- **Medical Core (MedicalProfile & Medication):** The MedicalProfile centralizes the patient’s physiological and clinical variables, such as age, allergies, and chronic diseases. It interacts with the Medication class, which serves as a model for active substances imported via external pharmaceutical APIs.
- **Analysis Engine (InteractionAnalyzer):** Operating as a service class, the analyzer orchestrates the business logic. It consumes data from the profile and the list of selected medications to query the integrated artificial intelligence.
- **Contextual Intelligence (AnalysisResult):** Unlike static database systems, this structure generates a dynamic AnalysisResult. The final report is not merely

the result of a chemical conflict, but an intelligent interpretation of risks based on the patient’s specific context, such as pre-existing conditions (e.g., Diabetic Retinopathy) or age-specific risks.

2.4 Conclusion of the Chapter

This chapter has established the formal foundation for MediCheck by translating the clinical needs identified in the State of the Art into a structured technical design. Through comprehensive requirements analysis, we defined the essential functional pillars—user authentication, medical profile standardization, and real-time interaction checking—alongside critical non-functional standards for security and performance. By modeling these requirements through Use Case and Sequence Diagrams, we mapped the dynamic interactions between the user and the system’s backend. Furthermore, the Class Diagram provided a structural blueprint, ensuring that the application’s business logic moves beyond simple data retrieval to deliver context-aware clinical analysis. This conceptual framework ensures that the subsequent implementation phase is built upon a stable, scalable, and patient-centered architecture.

Chapter 3

Realization and Technical Implementation

3.1 Introduction

This chapter details the technical implementation and realization of the MediCheck mobile application. After presenting the software environment and the specialized development tools used to build the solution, we will describe the global architecture of the application.

We will illustrate the core functional implementation by detailing the user interfaces and the business logic that drives the intelligent interaction analysis. Finally, we will present the final results achieved, followed by a discussion of the technical challenges encountered and the solutions implemented to ensure a high-performance clinical monitoring system.

3.2 Software and Technology Environment

The software ecosystem utilized for the development of MediCheck includes the following components:

- **Integrated Development Environments (IDE):** The primary environments used for coding and debugging were Visual Studio Code and Android Studio, providing a robust suite for Flutter development and emulator testing.
- **Flutter SDK and Dart Language:** The application was built using the Flutter framework for cross-platform mobile development, leveraging the Dart programming language to ensure high-performance UI rendering and reactive logic.
- **Version Control System:** Git was employed for source code management, with remote repositories hosted on GitHub to ensure development continuity and version tracking.

- **Backend Services (Firebase):** Firebase served as the core infrastructure, specifically utilizing Firebase Authentication and Cloud Firestore.

3.3 Global Architecture and Technological Choices

The MediCheck application is built on a modern client-server architecture. The “client” side consists of the mobile application, while the “server” is comprised of a suite of cloud services. This approach was selected for its flexibility, scalability, and its ability to simplify the development and maintenance of the backend infrastructure. By offloading server management to the cloud, more focus was placed on refining the intelligent analysis and user experience.

3.3.1 Application Mobile (Frontend)

The frontend of MediCheck is built using a modern stack designed for high performance and cross-platform reliability. Below are the core technologies that power the user experience.

Flutter Framework



Figure 3.1: Flutter Logo

Flutter is an open-source UI software development kit created by Google. It was chosen for this project because it allows for cross-platform development from a single codebase. Its “Hot Reload” feature accelerated the design of the medical forms and interaction dashboards seen in the previous chapters. Flutter’s widget-based architecture ensures that the UI remains consistent across both Android and iOS devices.

Dart Programming Language



Figure 3.2: Dart Logo

Dart is the client-optimized language that powers Flutter. It is designed for fast-performing apps and provides a smooth user experience with its sound type system and garbage collection. In MediCheck, Dart is used to handle the asynchronous logic—such as fetching drug data from external APIs and processing the AI analysis—without freezing the user interface.

Android Studio



Figure 3.3: Android Studio Logo

Android Studio is the official Integrated Development Environment (IDE) for Android app development, built by Google on top of JetBrains' IntelliJ IDEA. In the context of this project, it acts as the primary workspace where the code is written, tested, and compiled into a functional application, providing an essential suite of tools like emulators and debuggers to ensure system stability.

3.3.2 Back End Services



Figure 3.4: Firebase Logo

Firebase is a comprehensive Backend-as-a-Service (BaaS) platform developed by Google. It provides a suite of cloud-hosted tools that allow developers to build and scale applications without the need to manage physical servers or complex backend infrastructure. For MediCheck, Firebase acts as the secure backbone for data storage and user management.

The following core modules were utilized:

- **Firebase Authentication:** This service manages the entire user identity lifecycle. It provides secure sign-in and sign-up mechanisms, ensuring that each medical profile is private and accessible only to the authenticated owner.
- **Cloud Firestore:** This is a flexible, scalable NoSQL document database used to store and sync data in real-time. In this project, Firestore is used to host user medical profiles (age, weight, height) and their clinical history (allergies, chronic diseases, and surgeries). Its document-based structure allows for fast queries and seamless integration with the Flutter frontend.

3.4 General Project Structure and Functional Implementation

This section describes the realization of the MediCheck application. It first presents the software architecture, designed to ensure maintainability, testability, and scalability, followed by the technical implementation of its essential features.

3.4.1 General Organization and State Management

The development of MediCheck follows a modular architectural pattern, designed to decouple the user interface from the underlying business logic. This separation is essential for maintaining a clean codebase, allowing for independent testing of the clinical analysis engine without affecting the visual components.

The implementation focuses on three core layers:

- **The Presentation Layer:** This consists of the Flutter widgets that form the user interface. It is responsible for capturing user input—such as physical metrics and medication searches—and displaying the processed clinical results in a clear, readable format.
- **The Business Logic Layer:** This acts as the intermediary. It handles the logic for data validation, coordinates calls to external pharmaceutical APIs, and manages the communication with the AI model for interaction analysis.
- **The Data Access Layer:** This layer manages persistent information. It handles the asynchronous requests to Cloud Firestore for retrieving medical profiles and interacts with Firebase Authentication to secure user sessions.

To ensure the application remains responsive, State Management is utilized to handle the dynamic flow of data. By using reactive patterns, the application can instantly update the UI whenever a user modifies their medical profile or when the AI returns a new analysis report. This ensures a seamless, “real-time” feel, where data synchronization between the local device and the cloud backend happens invisibly in the background.

3.4.2 Authentication Implementation

Authentication secures access to the application by utilizing Firebase Authentication. This process ensures that each user has a private space for their medical data.

Registration and Login Algorithm

1. **Inputs:** Full name, Email, and Password. For new accounts, additional physiological data is required to complete the medical profile after the initial credentials are created.

2. **Registration Process:**

- a) The user enters their full name, email, and password.
- b) *Field Validation:* The system checks for a valid email format, password complexity, and ensures the password confirmation matches.
- c) *Account Creation:* A request is sent to Firebase Authentication to create a new unique user identity.

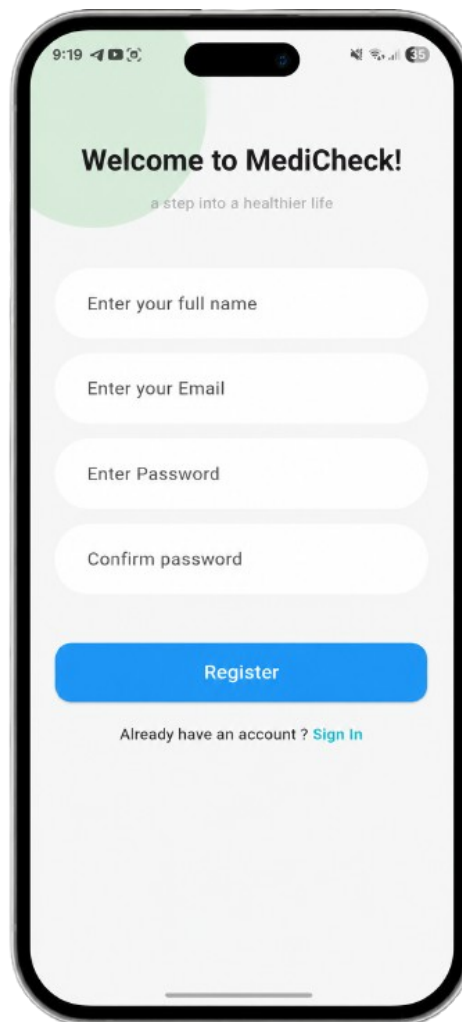


Figure 3.5: Registration Screen

3. **Login Process:** The login functionality allows returning users to securely access their existing medical records and analysis history.

- a) *Data Entry*: The user provides their registered email address and password.
- b) *Field Validation*: The application performs a local check to ensure the fields are not empty and that the email format is valid before sending a request to the server.
- c) *Firebase Authentication*: The system makes an asynchronous call to Firebase Authentication to verify the credentials.
- d) *Post-Authentication Routing*: Upon successful verification, the AuthWrapper component intervenes to manage the user's session:
 - It retrieves the user's unique UID.
 - It checks the Cloud Firestore database to verify the completion of the user's medical profile.
 - If the profile is complete, the user is immediately redirected to the Main Dashboard; otherwise, they are prompted to finish any missing clinical information.
- e) *Exception Handling*: In the event of incorrect credentials or network issues, the application displays clear feedback (e.g., "Invalid password" or "User not found") via a user-friendly snackbar or alert dialog.

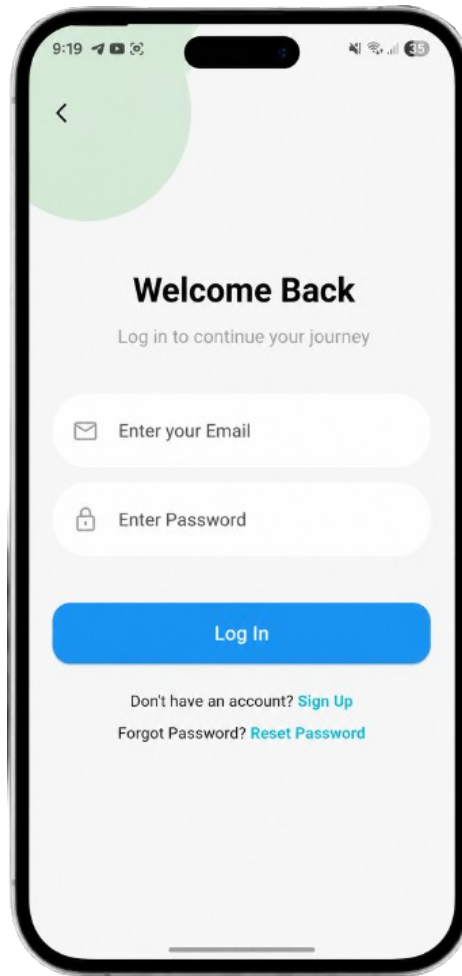


Figure 3.6: Login Screen

3.4.3 Medical Profile Implementation

After the initial account creation, the user is directed to the Medical Info screen. This module is designed to collect standardized health data, which serves as the foundational context for all subsequent drug interaction analyses.

Functional Breakdown

- **Physical Metrics Collection:** The user inputs their core biological data, including Age, Gender, Weight, and Height. These variables are essential because drug dosages and side effects often vary significantly based on physical build and physiological factors (e.g., pediatric vs. geriatric risks).
- **Dynamic Clinical Search:** To ensure data accuracy, the “Chronic Diseases,” “Allergies,” and “Surgeries” sections utilize a search-as-you-type mechanism. Instead of free-text entry, which could lead to typos or clinical ambiguity, this feature

queries a standardized medical database. This ensures that the terms stored in the profile are clinically recognized and compatible with the AI analysis engine.

- **Data Persistence (Firestore):** Once the user completes these sections, the data is structured into a JSON object and sent to Cloud Firestore. It is stored under a specific document linked to the user's UID. This ensures that the profile is persistent and can be retrieved instantly every time the user performs a new medication check.

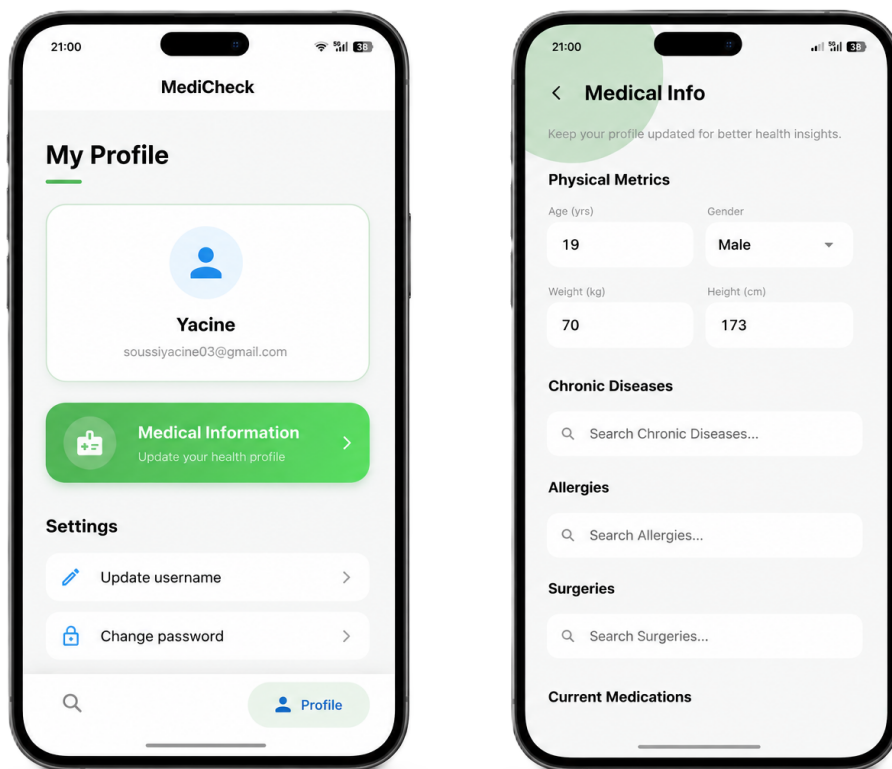


Figure 3.7: Medical Info Screen

3.4.4 Interaction Checker and Clinical Analysis Engine

The Interaction Checker is the central feature of the MediCheck system. It allows users to simulate the combined effects of multiple medications while simultaneously cross-referencing them with their personal medical profile.

Functional Implementation

- **Standardized Medication Search:** To maintain clinical integrity, the search bar utilizes the RxTerms/NLM API. As the user types, the system suggests standardized FDA-approved names (e.g., *Acetaminophen* instead of *Paracetamol*). This normalization is critical to ensure the AI engine correctly identifies the chemical compounds involved.

- **Dynamic Medication Management:** Users can add multiple medications to a “Selected List.” Each item is managed as a dynamic chip, allowing the user to easily review or remove medications before triggering the final analysis.
- **Contextual AI Analysis:** When the “Check Interactions” button is pressed, the application orchestrates a complex background process:
 - *Data Retrieval:* The app fetches the user’s latest physical metrics and medical history from Cloud Firestore.
 - *Prompt Construction:* A structured prompt is sent to the integrated AI Engine, containing the list of selected medications AND the user’s specific clinical context (e.g., “User has diabetic retinopathy”).
 - *Processing:* The AI relies on an existing list of interactions provided by the FDA database and then evaluates the drug-to-disease contraindications.
- **Clinical Result Presentation:** The output is displayed in the Clinical Analysis Result card, which includes:
 - *Severity Level:* (e.g., LOW, MODERATE, HIGH) providing an immediate visual warning.
 - *Reasoning:* A detailed, natural-language explanation of the risks. For example, if the user has an allergy to ibuprofen set in their medical profile, when aspirin is selected, the system shows a HIGH severity level due to aspirin containing ibuprofen.

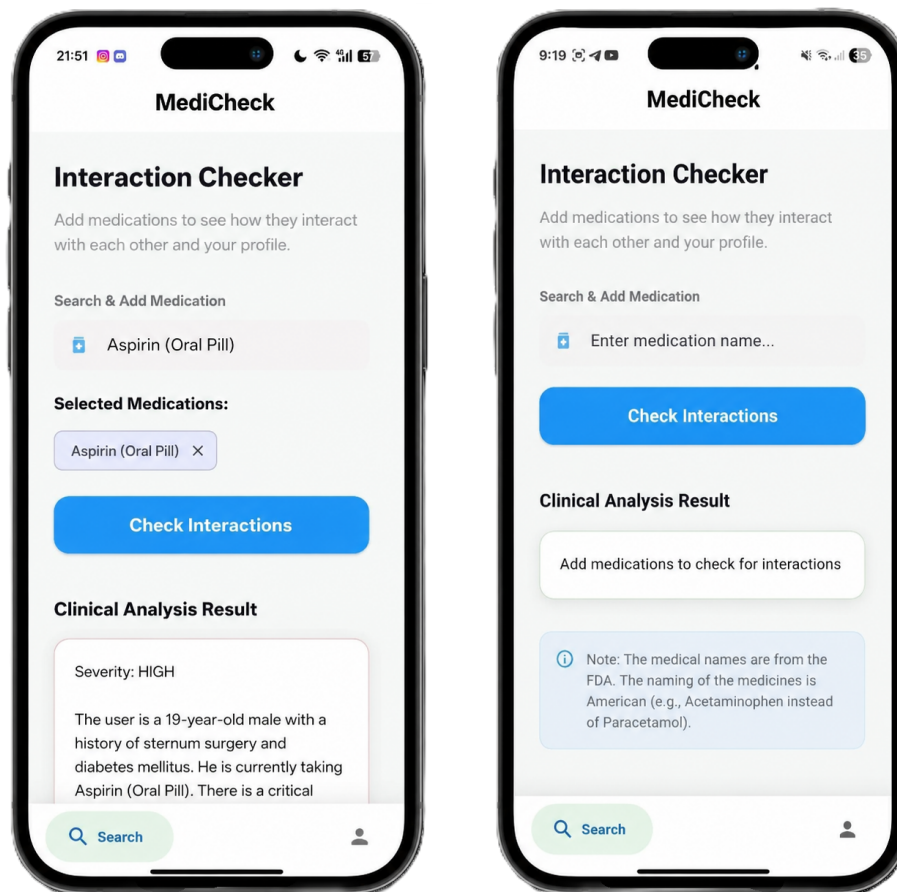


Figure 3.8: Interaction Checker and Clinical Analysis Screen

3.5 Conclusion of the Chapter

The realization phase of MediCheck successfully migrated the conceptual designs of the previous chapter into a high-performance, functional mobile solution. By leveraging the Flutter SDK and Dart, we achieved a responsive, cross-platform interface capable of handling complex asynchronous logic. The integration of Firebase provided a robust infrastructure for secure authentication and real-time data persistence through Cloud Firestore. The core technical achievement of this chapter was the implementation of the Clinical Analysis Engine, which effectively bridges the gap between raw pharmaceutical data and personalized patient context using integrated AI reasoning. Despite the challenges of normalizing medical terminology and managing state across modular layers, the resulting application provides a seamless experience for managing medication safety. This technical success sets the stage for future enhancements, such as OCR integration and broader health record connectivity.

General Conclusion

At the conclusion of this thesis, we return to our starting point: the critical need to integrate modern technology into the pharmaceutical care process. Driven by the observation of risks associated with adverse drug events and the complexity of personal medical histories, our primary objective was to design and develop an intelligent mobile solution capable of providing personalized clinical guidance.

This work resulted in the realization of MediCheck, a cross-platform mobile application developed with the Flutter framework. In accordance with our objectives, MediCheck provides users with an intuitive interface to:

- **Standardize Medical Identity:** By creating a digital medical profile that includes physical metrics, chronic diseases, and allergies using clinical standards.
- **Prevent Adverse Drug Interactions:** Through a real-time interaction checker that cross-references multiple medications with the user’s specific health context.
- **Leverage Artificial Intelligence:** By moving beyond static databases to provide intelligent, natural-language clinical reasoning that explains the “why” behind potential medical risks.
- **Secure Sensitive Health Data:** Utilizing a robust cloud infrastructure via Firebase to ensure the confidentiality and availability of patient records.

Throughout the chapters of this thesis, we first explored the state of the art to justify our technological choices. We then detailed the design phase, translating the complex needs of pharmaceutical safety into a robust software architecture. Finally, we demonstrated the concrete implementation of the application and the technical logic that drives its intelligence.

While this project successfully demonstrates the feasibility and relevance of an AI-driven pharmaceutical assistant, we recognize certain limitations. These include the current manual entry of medications and the potential for direct integration with electronic health records (EHR). These limitations provide stimulating perspectives for future evolution, such as scanning medication barcodes via OCR or integrating with official pharmacy systems in Algeria.

In conclusion, this work provides a concrete contribution to the modernization of healthcare in Algeria by proposing a patient-centered solution. MediCheck repre-

sents a significant step toward improving patient autonomy and safety, aligning with contemporary expectations for digital health and the national ambitions for e-health innovation.

Bibliography

- [1] Google. *Flutter Software Development Kit*. UI toolkit for building applications for mobile, web, and desktop from a single codebase. 2017. <https://flutter.dev/>
- [2] Google. *Firebase*. A comprehensive app development platform that serves as a Backend-as-a-Service (BaaS) for mobile and web applications. <https://firebase.google.com/>
- [3] Google. *Android Studio*. The official Integrated Development Environment (IDE) for Android app development. <https://developer.android.com/studio>
- [4] National Library of Medicine. *RxNorm*. A normalized naming system for generic and branded drugs in the U.S., produced by the NLM to facilitate interoperability between electronic health systems. <https://mor.nlm.nih.gov/RxNav/>
- [5] U.S. Food and Drug Administration. *openFDA Drug Adverse Event API*. Returns data collected from the FDA Adverse Event Reporting System (FAERS). <https://api.fda.gov/drug/>
- [6] Regenstrief Institute / NLM. *Clinical Tables – Medical Conditions*. A list derived from the Medical Gopher program containing over 2,400 medical conditions along with ICD-10-CM and ICD-9-CM codes. <https://clinicaltables.nlm.nih.gov/apidoc/conditions/v3/doc.html>
- [7] National Library of Medicine. *Clinical Tables – Surgical Procedures*. <https://clinicaltables.nlm.nih.gov/api/procedures/>
- [8] National Library of Medicine. *Clinical Tables – RxTerms (Allergies)*. <https://clinicaltables.nlm.nih.gov/api/rxterms/v3/>